



Chama

A non-custodial, peer-to-peer Bitcoin marketplace for local money — built on Nostr, Fedimint, and Lightning.

"Chama is a coordinator, not a wallet. Lightning is the universal interface in and out. Ecash exists only as the cryptographic substrate for the brief lock-claim window. Communities are the social layer that makes commerce trustworthy."



Seller — locks the
sats



Buyer — claims the
sats



Arbiter — breaks
ties

0 What this document is

A self-contained tour of Chama's architecture for a technically literate reader — enough to understand the whole system without reading the repository. It explains what Chama is, the three protocols it stands on, how a single trade ties them together, and then goes deep on the one piece that is being built right now: the **arbiter bond**.

1 · What Chama is	p.2
2 · The core idea: a coordinator, not a wallet	p.2
3 · Three protocols, and how they fit together	p.3
4 · The escrow: the Woven Trust mark & 2-of-3 Shamir	p.4
5 · The trade lifecycle (Option B)	p.5
6 · One trade, all the pieces at once	p.6
7 · Communities, not federations	p.7
8 · Reputation & the arbiter trust model	p.8
9 · The hard parts: failure modes & fund-safety doctrine	p.9
10 · The arbiter bond (being built now)	p.10
11 · Where it stands & what's next	p.13

1 What Chama is

Chama is a **peer-to-peer marketplace for local money**. Two strangers who don't trust each other can swap Bitcoin for fiat — M-Pesa in Nairobi, Orange Money in Dakar, a bank transfer in Lyon — or trade goods, pay each other's bills, or lend, with a neutral third party holding the stakes only as long as the deal is live. It is **non-custodial and serverless**: there is no company account holding anyone's money and no backend that can be subpoenaed, because there is no backend at all. The client is the whole product.

It runs as one codebase across several surfaces: the **web app** (the canonical surface, `chama.community / getchama.app`), a **Tauri desktop** binary, and an **Android APK** distributed through Zapstore and GitHub. The first real-world target is a **Nairobi launch**, with a francophone (Benin / Cameroon / Senegal) framing close behind. Version 4.0.0 shipped on 24 June 2026 and has already been used for live trades in Kenya — real Bitcoin out to M-Pesa in minutes.

The thing that makes it more than "another escrow app" is the choice of primitives. Most Bitcoin apps make the user learn the vocabulary of whatever cryptographic toy they're built on. Chama hides every primitive behind a single human-legible idea — your *community* — and uses three different protocols, each for the one job it is best at.

2 The core idea: a coordinator, not a wallet

This is the sentence everything else hangs from:

Chama is a coordinator, not a wallet. Money enters and leaves over Lightning; ecash exists only for the brief window a trade is locked; nothing idle is ever held.

There is **no balance screen** in Chama, by design. The mental model of "a wallet with money sitting in it" is deliberately dead. The lifecycle of every trade is:

BOLT11 in (funding) → ecash held in escrow only during LOCK→CLAIM → BOLT11 out (claim) →
local wallet drains to zero

Between trades, the local wallet sits at **zero**. Sats are visible only while they are actively committed to a specific trade, with a specific counterparty, heading toward a specific Lightning destination. The only money surface is an "**In escrow: 50,000 sats**" pill — never a balance. This is the product's deepest ethical commitment: Chama does not custody funds, the federation does not custody unspent funds, and the user's device does not store idle balances.

That commitment has a name and a doctrine, repeated in the codebase like a mantra:

THE CHAMA PROMISE — never lose a sat; never send one twice

Like a shepherd leaving the ninety-nine to find the one that is lost, Chama treats every stranded sat as a structural failure to repair, not an acceptable cost. Its twin: when a payout's outcome is *unknown* — a watch timed out, a connection dropped mid-settle — Chama **refuses to re-send** and reconciles to the durable record instead, because a second send is unrecoverable. **Unknown ⇒ confirm, not retry.**

Two design consequences fall out of this and recur everywhere in §5–§10. First, **funding is the lock** — the moment Chama sees the Lightning invoice paid, it mints the ecash and splits it atomically; there is no separate "lock" button, and the user experiences one event, not two. Second, the entire flow has only **two consent gates that move money: VOTE and CLAIM**. Everything else that older escrow designs treated as a step — join, ready, lock — collapses into a side-effect of payment landing.

3 Three protocols, and how they fit together

Chama doesn't invent its own network. It stitches together three existing Bitcoin-native protocols, each doing the single job it is best suited for. Understanding which job belongs to which protocol is most of understanding the system.

⚡ LIGHTNING — the universal interface (money in & money out)

Buyer funds with a BOLT11 invoice · winner claims to a BOLT11 invoice · this is the only rail value crosses

NOSTR

the nervous system

- identity (your npub)
- trade coordination events
- encrypted chat (NIP-44)
- ratings & arbiter pools

kinds 38100–38108, 38130 ...

FEDIMINT

the escrow vault

- client-side ecash (WASM)
- mints at fund time
- splits into 3 SSS shares
- redeems at claim time

backed by a guardian federation

CHAMA (the client)

the coordinator

- pure escrow state machine
- orchestrates the 3 protocols
- holds no funds, no server
- same code on web/desktop/APK

React UI over a Rust sidecar

No backend. No custodian. The three protocols meet only inside the user's own client.

Lightning bridges value · Nostr carries coordination & trust · Fedimint provides the escrow substrate

Figure 1 — Each protocol does the one job it is best at; Chama is the client that composes them.

Nostr — identity, coordination, and trust

Nostr is the messaging and identity layer. A user *is* their Nostr keypair (their npub); there are no usernames or passwords, and signing in means connecting a NIP-07 signer (nos2x, Alby, Amber) or running inside the Fedi mini-app. Every meaningful event in a trade — the listing, the lock, each vote, the claim, the chat messages, the ratings, the arbiter availability posts — is a **signed Nostr event** published to relays, in a custom set of "kinds" (38100–38108 for the escrow itself, with neighbours like 38130 for bonds and 38104 for arbiter pools). Sensitive payloads are encrypted with **NIP-44**. Because the whole trade is just a set of replayable signed events, any client can reconstruct the exact state of a trade by replaying them — order-independently.

Fedimint — the ecash escrow substrate

Fedimint is a federated Chaumian-ecash system: a set of *guardians* collectively run a mint that issues bearer ecash tokens. Chama runs the Fedimint client **in the browser as WASM**, entirely client-side. Its only role in Chama is to be the cryptographic material for the escrow: at fund time the client mints ecash, and — crucially — it then splits that ecash using **Shamir Secret Sharing** into three shares that no single party can reconstruct alone. Fedimint is the vault; it is never a balance.

Lightning — the universal in/out interface

Lightning is how value actually enters and leaves. The buyer funds a trade by paying a **BOLT11 invoice**; the winner claims by pasting a BOLT11 invoice that the reconstructed ecash is redeemed into. This is the quiet genius of the design (see §6): because money always crosses on Lightning and never as ecash, two people on *different* federations can trade with no friction at all — one federation mints, the other ingests, and Lightning bridges the gap.

THE FOUR LAYERS OF THE CLIENT

Internally the code is four clean layers: **(1)** a *pure* escrow state machine — (state, event) → state, no I/O, fully unit-tested; **(2)** the relay layer — multi-relay WebSockets, NIP-44, signers; **(3)** the Fedimint WASM client — ecash mint, SSS split/combine; **(4)** the React UI. On desktop and Android the Fedimint work runs in a compiled **Rust sidecar** bridge rather than WASM, but the protocol semantics are identical — the UI never forks.

4 The escrow: the Woven Trust mark & 2-of-3 Shamir

Every trade has exactly three roles, and the brand encodes them at the deepest level. The **Woven Trust** mark — three rounded ribbon bands (orange, teal, purple) woven over and under one another like a trust knot, with a small orange reputation spark in the negative space at the center — is not decoration; it is the trust model drawn as a glyph: three participants woven into one accountable structure. When the weave is whole, the trade is whole.



SELLER — Bitcoin Orange

Locks the sats. Holds SSS share #1.

BUYER — Nostr Purple

Claims the sats. Holds SSS share #2.

ARBITER — Signal Teal

Breaks ties. Holds SSS share #3. Gated until a dispute.

3 shares minted · any 2 reconstruct the ecash · no 1 party can act alone

Figure 2 — A 3-of-3 split with 2-of-3 resolution. The woven mark *is* the trust model: three loops, one knot.

Mechanically: when a trade locks, the freshly minted ecash is split into **three Shamir shares** with a threshold of two. Share #1 is encrypted (NIP-44) to the **seller**, share #2 to the **buyer**, share #3 to the **arbiter**. To reconstruct the ecash and pay it out, you need **any two of the three** shares. This produces the entire trust model with one piece of math:

- **Happy path** — buyer and seller agree (e.g. "I sent the fiat" / "fiat received"). Their two shares reconstruct the ecash and release it to the winner. The arbiter is never involved.
- **Dispute** — buyer and seller disagree. Now the arbiter's share becomes the tiebreaker: arbiter + one party = two shares = resolution. The arbiter can only *refund* (return to the locker), never invent a new recipient.
- **No unilateral theft** — no single party, including the arbiter, ever holds enough to move the money alone.

HOLDER-ONLY SHARES (a hard invariant)

The LOCK event seals exactly one share per recipient, and **the application layer itself cannot reconstruct** — only the holders' keys can. Roles are *derived from the event signer*, never self-asserted: you can't claim to be the buyer, you can only sign as the key the buyer's share was encrypted to. Votes are immutable and one-per-pubkey. A heal can only ever REFUND, never flip a release. These are enforced in the state machine and pinned by tests — they are the sacred core the bond work (§10) must reuse without weakening.

5 The trade lifecycle (Option B)

The internal name for the lifecycle is "**Option B**": atomic funding, no persistent balance. Here is a complete trade, end to end. The diagram shows the money path; the prose explains the consent gates.

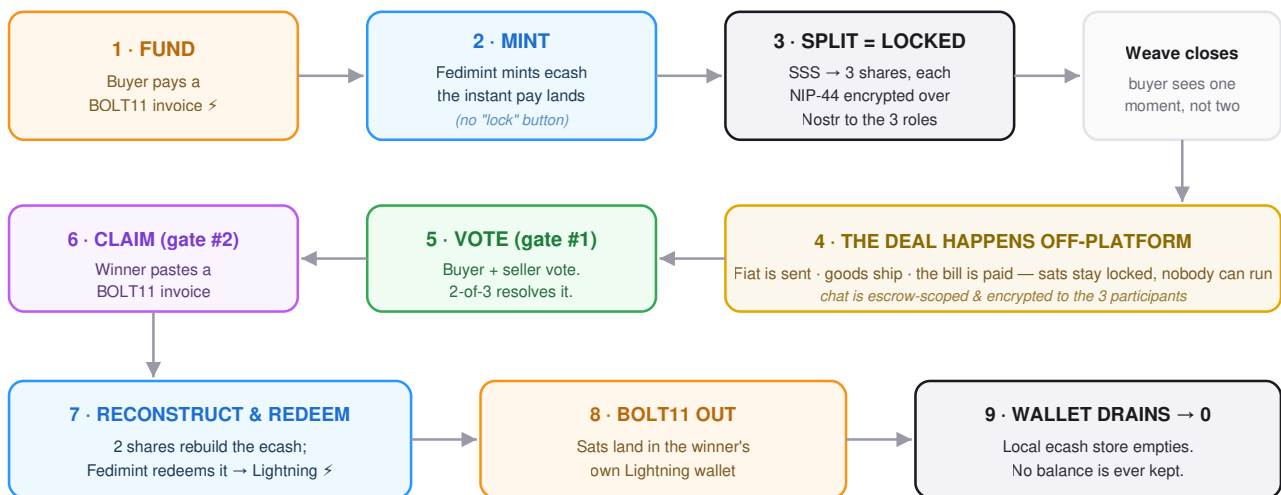


Figure 3 — Option B: BOLT11 in → ecash escrow → BOLT11 out → drain to zero. Only steps 5 and 6 ask the user for consent.

The same spine carries every kind of trade

That nine-step spine is "vertical-agnostic." Four trade types ride it, changing only labels and a few per-listing fields:

Vertical	What it is	Who locks	Vote labels
Market-place	Buy goods, services, or digital files with sats	Buyer (of goods)	"I received it" / "Item delivered"
P2P	Swap sats for fiat over a mobile-money / bank rail	Seller of sats	"I sent the fiat" / "Fiat received"
Bill Pay	A volunteer pays your fiat bill, earns sats	Bill-ower (of sats)	"Bill has been paid" / "My bill was paid"
Lending	Two stacked Option-B cycles: loan, then re-payment	Lender, then borrower	per loan terms

Bill Pay is the strategically clever one: it is a stealth onboarding funnel. A newcomer earns their *first* sats by paying someone else's M-Pesa or Revolut bill — no exchange, no KYC, no "buy Bitcoin" friction. Bitcoin arrives as the by-product of a relationship favour ("I'll give you your first bitcoin if you pay this bill of mine"), which is socially legible to anyone, Bitcoin-pilled or not.

6 One trade, all the pieces at once

Walk a single P2P trade and watch the three protocols hand off to each other. Alice in Nairobi wants to sell 50,000 sats for KES via M-Pesa; Ben wants to buy.

1. **Listing & identity (Nostr).** Alice publishes a signed listing event. She and Ben are just npubs with rating histories — no accounts. The listing advertises the *rail* ("M-Pesa") but not her phone number; private handles stay private until trust is committed.

2. **Funding (Lightning → Fedimint).** As the sats-seller, Alice locks. She pays a BOLT11 invoice; the instant it lands, her in-browser Fedimint client mints 50,000 sats of ecash. *Funding is the lock* — there is no second step.
3. **The split (Fedimint + Nostr).** That ecash is immediately Shamir-split into three shares. Share #1 → Alice, #2 → Ben, #3 → an arbiter the system auto-assigned from the community pool. Each share is NIP-44-encrypted and published as the LOCK event. The Woven Trust mark closes into one knot on everyone's screen.
4. **The deal (off-platform + encrypted chat).** Ben sends KES to Alice's M-Pesa number (now revealed to the three participants only). They coordinate in escrow-scoped, encrypted Nostr chat.
5. **Resolution (Nostr → Fedimint → Lightning).** Both vote "done." Two shares (Alice's + Ben's) reconstruct the ecash with no arbiter needed. Ben — the winner of the sats — pastes a BOLT11 invoice from his own wallet; the ecash is redeemed and the 50,000 sats route to him over Lightning. Both local wallets drain to zero.

THE GENIUS HIDING IN STEP 5 — cross-federation trading is free

Because value enters and leaves on **Lightning**, never as ecash, Alice and Ben do *not* have to be on the same federation in steady state. One federation mints at fund time; the winner's federation ingests at claim time; Lightning bridges the two. No ecash ever crosses a federation boundary (which would strand it). This is why Chama can present "communities" instead of "federations" to users — the plumbing genuinely doesn't require them to match for value to move. (The one hard rule: a single trade's three participants must share a federation *for that lock*, because the shares are minted on one fed. Chama enforces this by silently switching the client to the listing's federation — trivial, because the wallet is always at zero between trades.)

7 Communities, not federations

This is the deepest user-facing simplification, and it's worth its own section because it explains a lot of naming you'll see in the code. **The word "federation" appears nowhere in the normal UI.** Users pick a **Community** once, at sign-in, and a community is defined as:

currency-primary, country-and-language-multivalent. One currency always (the load-bearing axis). One or more countries (CFA spans 14; the euro spans 20). One or more languages. And a cultural identity — "where my family lives" — compressed into a single flag glyph.

Behind each community sits some Fedimint federation, chosen automatically and never surfaced. Kenya might map to Afribit or Bitsacco; the US to a federation like GBF; and any country without its own community-run federation falls through to **Bitcoin Life Federation (BLF)**, the universal fallback — while still keeping its own flag (Senegal · CFA, never "BP"). New federations onboard permissionlessly, like Nostr relays: a guardian set pastes its invite, claims a flag, and the community is listed. The flag is the headline; the federation is the footnote.

The principle underneath, repeated in the philosophy doc, is **"every user has a home."** A Chama user is never left in an identity-less limbo. Their home community is sticky — every reconnect, recovery, and cold-start routes them back to it. The current multi-federation set includes BP, Afribit Kibera, Bitsacco, BLF, and GBF, and a standing invariant is simply: *never break fed-switching*.

User-facing word

What it really is (internal / advanced)

Community	Fedimint federation
In escrow	SSS-split ecash held during LOCK→CLAIM
Active funds in escrow	Local (OPFS) ecash balance — only ever non-zero mid-trade
Rating	NIP-44-encrypted Nostr reputation event
Arbiter	Holder of SSS share #3, the 2-of-3 tiebreaker

8 Reputation & the arbiter trust model

Reputation is not a feature bolted on the side — it is the **structural backbone**. Every completed trade emits rating events (a custom Nostr kind). Public aggregates (count, % positive, breakdown by vertical) sit on every npub's profile; individual comments are NIP-44-encrypted to the recipient. This one dataset is the fuel for *graduated authority* across the whole product: arbiters graduate (auto-assigned → manually pickable → community-elected), and merchants graduate (regular seller → recurring-payment-eligible). Earned authority replaces gatekept authority. The recurring question for any new feature is: "can the rating primitive already provide the trust signal, instead of inventing a new gate?"

Why the arbiter is the delicate part

The arbiter holds the tiebreaker share. That is **real power**, and the design refuses to paper over it. Two layers defend against arbiter abuse:

1. **Community election (social layer)**. Each community elects its own arbiter pool via Nostr availability events (`kind:38104`) — not a hardcoded list, not the federation's guardians. v1 ships a bootstrap pool; v2 replaces it with live elections. (Merging federation-guardian trust with arbiter trust is explicitly forbidden — it would concentrate two different kinds of power in one key.)
2. **Random assignment (protocol layer)**. Once elected, an arbiter is assigned to a trade by the system, not chosen by the parties. Buyer and seller cannot route a trade to a friendly arbiter, and the selection strips their own pubkeys out. Targeted collusion would require both election fraud *and* a lucky random draw.

The state machine already enforces a lot: the same pubkey can't fill two roles in one trade (`DUPLICATE_PARTICIPANT`); only pool members can serve (`ARBITER_NOT_IN_POOL`); buyer/seller keys are excluded from selection. What protocol *cannot* prevent is two humans with separate keys agreeing to collude off-platform. The honest answer the design gives is: a vetted, publicly visible arbiter **burns all accumulated reputation to steal one trade**, and at scale that is economically irrational.

Arbiter fees — paying for duty, not outcomes

Fee	Amount	Who pays	Why
Ambient	0.5% on <i>every</i> completed trade	built into the trade	the cost of having a trusted human available and invested — paid even when never called
Dispute escalation	+1.5% when a dispute is opened	split 0.75% / 0.75% between both parties, <i>regardless of who wins</i>	deters frivolous disputes; the arbiter earns more only because the work got harder, never because a side won

Total arbiter take on a disputed trade is 2%. The fee is fixed and non-negotiable — neither party can bid it up or down, which keeps the arbiter's incentive honest. The only lever an arbiter has to earn more is to attract more volume to their community.

THE GAP THIS SETS UP

Notice what is *missing*: the fees above are, today, essentially **cosmetic** — they are published and validated, but there is no economic stake making collusion irrational beyond reputation. The security audit literally flags this as "economic deterrents: social-only." Closing that gap — giving arbiters real skin in the game — is exactly what the **arbiter bond** is for. That's \$10.

9 The hard parts: failure modes & fund-safety doctrine

A serverless, client-side, three-protocol money app has sharp edges. Two are worth understanding because they're easy to confuse, and because the discipline around them is the same discipline that governs the bond.

Two different "can't reach the federation" failures

	① Fresh-join hang (discovery)	② Funding gateway (Lightning)
What fails	Joining a federation hangs while trying to <i>discover</i> its guardians.	Funding errors with "no reachable Lightning gateway... timed out."
Root cause	The native bridge resolves guardians via DNS + mainline DHT, which mobile networks & CGNAT throttle or block.	Guardians <i>were</i> reachable, but the federation's Lightning gateway is offline/absent (e.g. a dead <code>iroh://</code> gateway eating the timeout budget).
The fix	Add pkarr-over-HTTPS resolution (additive on top of DNS+DHT). Only reproduces on real mobile — localhost can't validate it.	Select a reachable gateway explicitly (clearnet-first probe, last-good cache) instead of blindly picking the first one.

Both shipped in v4.0.0. The reason to spell them out: they look identical to a user ("it won't connect") but live in completely different layers — one is *name resolution*, the other is *payment routing* — and conflating them sends you fixing the wrong thing.

The fund-safety doctrine, made concrete

The "never lose a sat, never send one twice" promise from §2 is not a slogan; it's a set of coded guards, and it is **platform-blind on purpose** — a guard that protects browser users but lies dormant on mobile is treated as a fund-loss *bug*, not a safety feature. A few examples of how seriously this is taken:

- **Fail-closed journaling.** The payout journal is checked *writable before* a payment is attempted; if it can't persist the record, the payout is forced into a "confirming" state rather than risking a blind re-send.
- **Unknown ⇒ confirm, not retry.** If a payout's outcome is ambiguous (a watch timed out mid-settle), Chama reconciles against the durable operation instead of re-sending, because the counterparty may already hold the sats.
- **No wipe on unknown balance.** The local ecash store is never wiped unless the balance is provably zero or provably clientless; "unknown" means refuse and ask.
- **Stranded notes are surfaced, never silently eaten**, with an export/escape hatch.

This matters for what comes next, because the bond is the **first time Chama locks real ecash into long-lived, multi-party custody** — and it inherits every ounce of this paranoia.

10 The arbiter bond — being built now

This is the piece your friend is most likely curious about, and the piece under active construction. It is described as "**the single highest fund-risk piece of work in Chama**," because it is the first time real ecash gets locked into long-lived, multi-party custody rather than a short trade escrow.

10.1 — Why bonds exist

Recall the gap from §8: arbiters wield the tiebreaker, but their only real disincentive against collusion is reputation. A **bond** gives an arbiter *skin in the game* — real locked Bitcoin that backs their authority. The more an arbiter has bonded, the larger the trades they're trusted to adjudicate. It turns "trust me, I have a good rating" into "trust me, I have capital locked that I lose if I misbehave."

10.2 — The reframe that drives the whole plan

There is a tempting shortcut, and the design explicitly refuses it. A bond *declaration* already exists as a Nostr event (`kind:38130`) — a signed claim, "I stake B sats." There is also a finished, unit-tested **exposure cap** ledger that would let the system say "this arbiter's bond is big enough to cover this trade." Flipping the switch to enforce it (`BONDS_ENFORCED = true`) is literally a one-liner. **And doing that now would be security theatre.**

WHY THE EASY VERSION IS A TRAP

A `kind:38130` declaration today is **unbacked** — a signed claim with *nothing actually locked*. An attacker could declare `bondMsats = 10,000,000`, instantly read as "Gold" tier, and unlock million-sat trades **with zero capital at risk**. So the keystone is *not* the enforcement flip — it's building **real custody** so that a declaration is backed by ecash that is genuinely locked. Enforcement must not flip until "I staked B" means "B is provably locked."

Hence the work is staged so that **no participant's money ever depends on an unproven custody loop**:

Phase	What gets built	Enforcement
2A	Real bond custody, the bonding ceremony, the heal/relock lifecycle, and the loud heal-prompt. Proven live with the trio's own tiny, short seed bonds on the real BLF federation.	OFF — no participant funds depend on it yet
2B	Wire the capacity context into arbiter assignment and flip enforcement on — but counting a bond <i>only</i> if it's backed by a verified live lock.	ON — caps now bind real, backed capital

2A is shippable on its own: its payoff is a **visibly bonded cabinet** — real money locked, legible at a glance ("this community is backed by X sats"). 2B turns that from observational into binding.

10.3 — How the bond actually works: reuse the 2-of-3, relabelled

The elegant move at the heart of the design: **a bond is just the trade escrow construct, relabelled**. Instead of inventing new custody crypto (which would be the most dangerous possible place to be original), a bond is the exact same audited **2-of-3 Shamir lock** from a normal trade — with the three roles renamed:

[buyer, seller, arbiter] → [owner, custodian A, custodian B]

A bond = a 2-of-3 SSS lock owned by the arbiter, custodied by two cabinet members

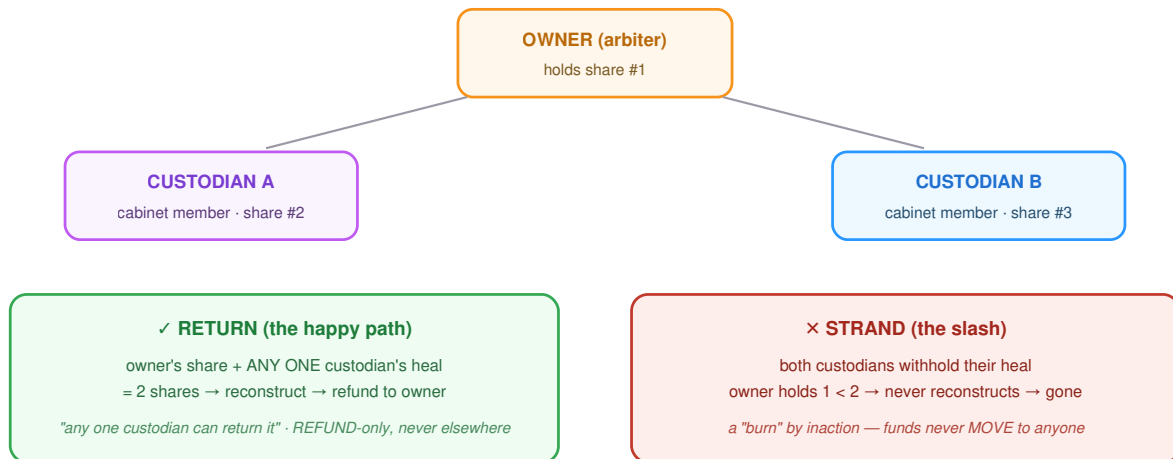


Figure 4 — The bond reuses the trade escrow's exact 2-of-3 lock, with zero new crypto.

This buys the entire bond mechanism with **zero new cryptography**:

- **Return** = owner's share + *either* custodian's heal-vote = 2 shares $\Rightarrow$ reconstruct $\Rightarrow$ refund to the owner. Any one custodian can give the bond back at term-end.
- **Strand (slash)** = both custodians withhold $\Rightarrow$ owner alone holds 1 < threshold $\Rightarrow$ the bond never reconstructs. This is the punishment, and its shape matters enormously (next).
- For a **cabinet member's own bond**, the two custodians are the *other two* cabinet members — each member's bond is held by the other two, returnable by any one of them.
- For a **general arbiter's bond**, the two custodians are two cabinet members (e.g. the federation owner + the community owner).

10.4 — The fund-safety invariants the bond must not break

The bond is fund-critical, so it inherits the §9 doctrine and adds a few of its own. These are non-negotiable:

BOND INVARIANTS

1. **2-of-3 is never weakened.** No bond mechanism may let one party hold two Shamir slots.
2. **Healing is REFUND-only to the owner.** A custodian healing a bond can return it to the owner and *nowhere else* — there is no path for a custodian to redirect a bond to themselves. (This reuses the existing `INVALID_HEAL_OUTCOME` guard.)
3. **A strand never MOVES funds.** Slashing is *delisting + a public record + withholding the heal*. There is deliberately no "seize" or "burn-to-treasury" code, because that would rebuild the very "a third party can move your funds" capability the escrow exists to forbid. Stranded sats benefit *no one* — the deterrent is the owner's loss, not anyone's gain.
4. **Enforcement is consent-layer, never a reducer reject.** An over-capacity arbiter is *skipped at assignment*; a hand-seated one triggers a warn-and-acknowledge. The state machine always accepts a valid LOCK — a live trade can never freeze because of capacity drift, and the assignable pool is guaranteed never-empty.

10.5 — The lifecycle: ceremony, term, return, relock

Phase 2A builds the full loop and proves it live before any enforcement exists:

- **Bonding ceremony.** Two flavours. The *cabinet seed ceremony*: the trio (the bootstrap cabinet) locks **simultaneously**, each member's bond held by the other two; when all three locks finalise, the community flips to "Live" on the globe. The *individual "post a bond"* surface lets any arbiter pick a term + amount and lock to two cabinet custodians — opt-in, to raise their exposure cap above the free floor, never a turnstile (entry stays free).
- **Term-end return (happy path).** Any one custodian casts the refund-only return heal; the owner reconstructs; the bond comes back. A bond is a distinct lock class that is **excluded from the normal trade auto-refund** — term-end must be a *deliberate* return, never an automatic one.
- **Slash + relock.** Slashing is delisting through a challenge window (due process for a delisting, not a confiscation court). A normal cabinet swap is bond-returned *before* re-locking the new trio, so no funds are ever at risk during a roster change; only a true emergency (lost key, rogue member) needs an off-cycle heal.
- **The loud heal-prompt (a UX invariant).** Returning an honest peer's bond is the single action that *protects* a peer, and it must be the **loudest prompt a cabinet member ever gets** — impossible to miss. Ignoring a loud, provable, in-term return request is then visibly *choosing* to strand a peer, which is the reputational deterrent against lazy or malicious withholding.

10.6 — The risks designed-for (not discovered)

Risk	Mitigation
Long-lived ecash inherits guardian risk. A bond is locked for weeks/months; a federation incident <i>during</i> a term could lose bonded capital ("long-lived ecash dies with its mint").	Bound it by sizing : first live bonds are tiny and short. Term length is a deliberate risk dial, not a detail.
Accidental strand of an honest bond. If the custodians who could heal are unavailable at term-end (lost keys, offline), an honest arbiter's bond also strands.	The loud heal-prompt + a generous return window. Cabinet key hygiene is treated as load-bearing.
Theatre risk. Enforcing caps on unbacked declarations.	2B counts a bond toward capacity only if it's backed by a verified live lock — never a mere declaration.

10.7 — Display bands & the honest framing

Bonded capacity surfaces as simple bands — **Bronze / Silver / Gold** (Gold ≥ 1M sats) — with honest copy ("backed by X sats; covers trades ≤ X"). And the design is careful to state that a bond is *not* the whole wall: bond + ratings + identity + the 2-of-3 escrow all **compose**. No single one is the complete defence; the bond just adds the economic leg that §8 was missing.

DECISIONS LOCKED (24 June 2026)

Ship **2A first** · prove it with a **trio-only tiny/short seed bond** on the real BLF federation · **short first term** (the guardian-risk dial) · reuse the **2-of-3** lock (owner's own share + 2 cabinet custodians), not a new threshold scheme. The lowest-fund-risk path on every axis. The seed test is the gate: lock → declaration references the

lock → term ticks → loud prompt fires → one custodian heals → owner reconstructs → relock — *and* a deliberate strand of a throwaway bond to prove it harms no one else.

11 Where it stands & what's next

v4.0.0 shipped on 24 June 2026 and is live on Zapstore, verified with real trades in Kenya (Bitcoin out to M-Pesa in minutes). That release carried the fund-safety core: the funding-gateway fix, the native fresh-join pkarr resolution, payout double-pay guards, relay resilience, and the first fiat off-ramp (Tando, M-Pesa via a Lightning address).

The near roadmap:

Re-lease	Theme	Contents
4.1.0	"newbie-polish" (next)	In-app help, ratings shown in chat, a fixed-rectangle trade view with a "You owe KES X" headline, a Kenya bill-type picker, an unread-chat badge — low-risk UI, shipped <i>before</i> the bond.
4.2.0	"finish the bond" (§10)	Phase 2A custody + ceremony + lifecycle (enforcement off), then 2B once the live loop is proven.

Further out, the philosophy doc sketches: manual then community-elected arbiters ("power to the Chamacitos"), recurring payments for graduated merchants, a 3D-globe community picker ("Hey Chama, where's home?"), self-hosted Lightning addresses (`you@chama.community`), and reputation portability so Chama ratings render in other Nostr clients. All of it sits on the same primitives described here — no new custody, no balance, no surfacing of "federation," and the rating system as the backbone for every new grant of authority.

Compiled from Chama's internal design corpus — the product philosophy, the consensus & security invariants registry, the architecture README, and the Phase-2 arbiter-bond design brief — as of 25 June 2026 (v4.0.0). Role colours, the Woven Trust mark, and the language conventions follow the project's brand spec. This document paraphrases that corpus for an outside technical reader; the code and the linked design docs remain the source of truth.